

PROJECT AND TEAM INFORMATION

Project Title

SmartFileVault: Automated File Compression and Storage Management

Student / Team Information

<i>Team Name:</i> <i>Team ID:</i>	<i>Avengers</i> <i>DSCPP-III-2026-T005</i>
Team Member 1 (Team Lead)	<i>Kartik Chaudhary</i> <i>Student ID: 2512520007</i> <i>kartikchaudharyshadow@gmail.com</i> 
Team Member 2	<i>Akshat Garg</i> <i>Student ID: 2510012614</i> <i>madhavakshat@gmail.com</i> 
Team Member 3	<i>Astha Negi</i> <i>Student ID: 2510370143</i> <i>asthanegi.001@gmail.com</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

- As users keep adding data over time modern systems keep collecting documents and many other kinds of files. A lot of these files sit unused for a time because people do not use them regularly yet they keep them just in case. When a file that has not been used for a while is required later the user must find it manually. Compressing it can take a lot of time and it becomes very inconvenient. To solve this problem we have SmartFileVault. It automatically spots files that have stayed unchanged for a period and compresses them to save space. SmartFileVault keeps track of file information so that compressed files can be easily restored and accessed whenever needed.

State of the Art / Current Solution (1 pt)

- Operating Systems already give you tools to clean up storage. People can use ZIP tools to shrink files manually which is time consuming. You can put used files in cloud storage but that needs an internet connection and more space. Some systems can manage storage automatically by looking at how files are used but they usually do not join this with a simple local file manager.
SmartFileVault does it all: it finds files that have not been touched for a while compresses them locally and lets you bring them back when you need them, so you save space yet keeping the data intact.

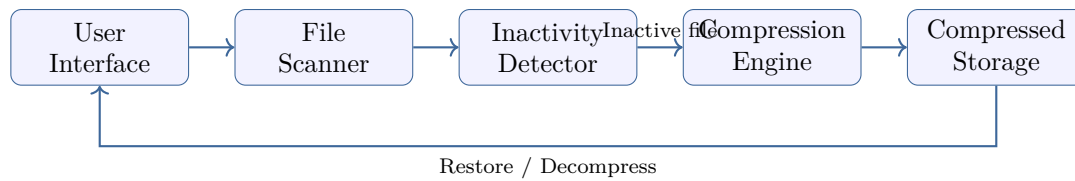
Project Goals and Milestones (2 pts)

- The main aim of the project is to study the concepts of file handling and compression techniques . The file handling process will be designed using C that will include file scanning to detect unused files. The project will also include a compression and decompression module based on zlib and will provide safe storage and restoration of the compressed files. Finally the entire system will be tested to ensure that the system works perfectly.

Project Approach (3 pts)

- At the start the system will scan the folder that has been selected. It will gather details such as the file name, the file location, the file size and the time that the file was last accessed. The user will set a time limit so that the system can find the files that have not been used during that period of time. Those files found by the system will be compressed using zlib. The system will keep the file information so that the files can be restored later. C will be used for the file operations and, for the data structures. C++ will be used for the application logic. The user interface will be built with HTML, CSS and JavaScript. The system will be tested with file sizes and time limits. The tests will check how much storage is saved, how well the compression works and whether the restoration is successful.

System Architecture (High Level Diagram) (2 pts)



System Workflow

- The user starts by selecting the desired folder through the interface. The file scanner gathers information about the files, in that folder and the inactivity detector then looks at the access date to spot the files that have not been used within the set period. The files that are thus identified are then stored in compressed form. Each time the user needs a file the system decompresses the file and restores it to its original usable state.

Project Outcome / Deliverables (1 pt)

- A working prototype will be produced which is capable of automatically identifying inactive files and compressing them with the use of the zlib library; this will free up storage space while at the same time keeping the files safe and allow them to be easily restored when required. The project will also include a simpler and more user-friendly interface for scanning and compression.

Assumptions

- The system takes it for granted that the user has been given the necessary permission to access or change the files or folders. Secondly, it assumes that the operating system will supply the required file information such as the last access time—in order to detect inactive files. The files which are to be compressed must not be altered during the compression process. The system will keep the necessary file information so that the compressed files can be recognized and restored when required. It is also assumed that enough storage space, the necessary permissions and the required libraries are available for both compression and decompression.

References

1. zlib Documentation – Compression and decompression library for C.
2. C++ Standard Library Documentation – File handling and object-oriented programming concepts.
3. C Programming Language References – File handling, structures and pointers.
4. HTML, CSS and JavaScript Documentation – Frontend development concepts.
5. CMake Documentation – Build configuration and project management.